

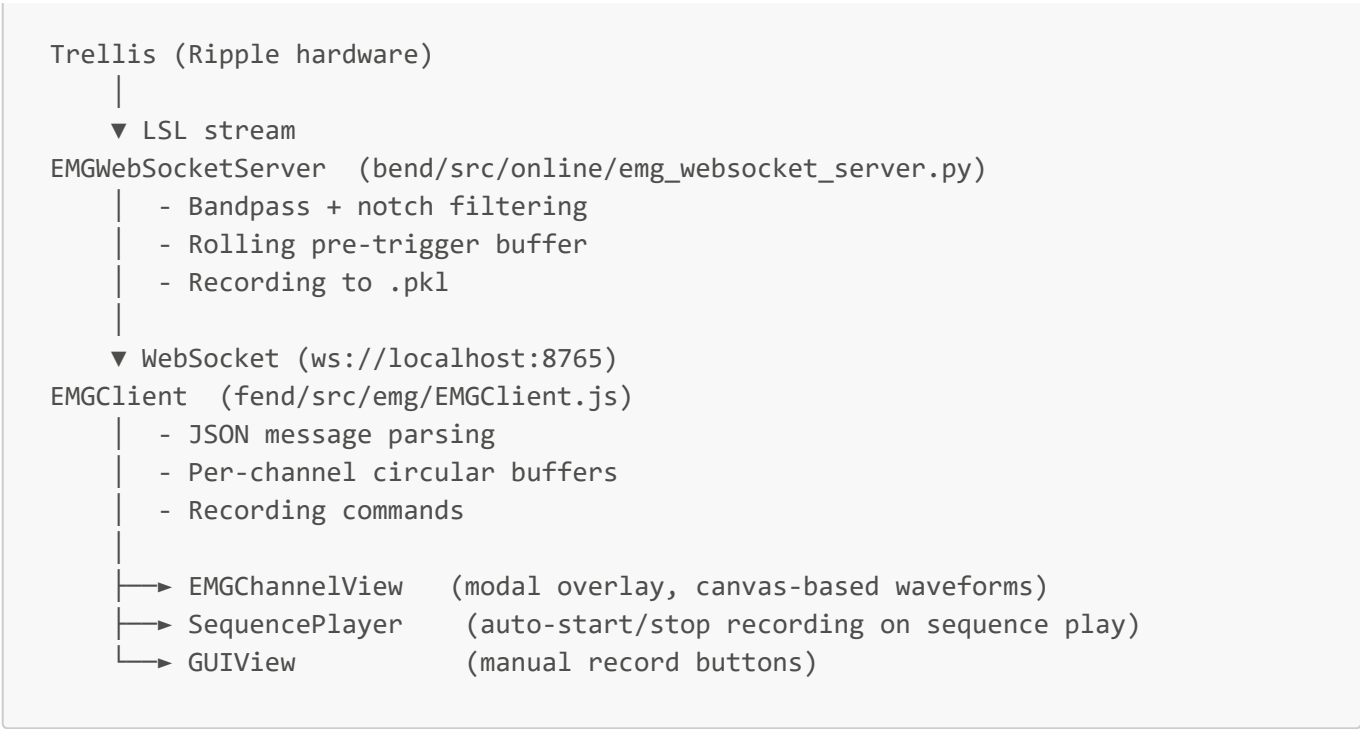
EMG Channel System

Technical reference for the frontend EMG subsystem: real-time data acquisition over WebSocket, channel buffering, recording lifecycle, and waveform visualization.

Table of Contents

- [EMG Channel System](#)
 - [Table of Contents](#)
 - [Architecture Overview](#)
 - [Module Inventory](#)
 - [EMGClient](#)
 - [Constructor Options](#)
 - [Connection Lifecycle](#)
 - [WebSocket Protocol](#)
 - [Server → Client](#)
 - [Client → Server](#)
 - [Channel Buffering](#)
 - [Recording API](#)
 - [startRecording\(metadata?\)](#)
 - [stopRecording\(timeline?\)](#)
 - [Event Callbacks](#)
 - [EMGChannelView](#)
 - [Rendering Pipeline](#)
 - [Canvas Dimensions](#)
 - [Pagination](#)
 - [Visual States](#)
 - [Integration Points](#)
 - [App.js \(Initialization\)](#)
 - [SequencePlayer \(Automatic Recording\)](#)
 - [GUIView \(Manual Controls\)](#)
 - [ProgressSidebar](#)
 - [Data Flow](#)
 - [Recorded Data Format](#)
 - [Configuration Reference](#)
 - [Server CLI Arguments](#)
 - [Frontend Defaults](#)
 - [Error Handling](#)
 - [Known Constraints](#)

Architecture Overview



The backend ingests raw EMG samples from a Ripple Trellis neural interface via Lab Streaming Layer (LSL), optionally applies a 20–500 Hz bandpass and 50 Hz notch filter, then pushes data to all connected WebSocket clients as JSON chunks every ~50 ms. The frontend `EMGClient` deserializes these chunks into per-channel `Float32Array` circular buffers, and `EMGChannelView` renders them on a `<canvas>` at display refresh rate.

Module Inventory

File	Role
fend/src/emg/EMGClient.js	WebSocket transport, channel buffering, recording commands.
fend/src/emg/EMGChannelView.js	Canvas-based waveform overlay with pagination.
bend/src/online/emg_websocket_server.py	Python server bridging LSL → WebSocket with filtering and recording.

EMGClient

Stateful WebSocket client that maintains a persistent connection to the backend EMG server.

Constructor Options

Option	Type	Default	Description
host	string	'localhost'	Server hostname.
port	number	8765	Server port.
autoReconnect	boolean	true	Reconnect automatically on disconnect.
reconnectInterval	number	3000	Milliseconds between reconnection attempts.

```
const client = new EMGClient({
  host: 'localhost',
  port: 8765,
  autoReconnect: true,
});
client.connect();
```

Connection Lifecycle



Reconnection: On unexpected disconnect, the client schedules a reconnection after `reconnectInterval` ms. The timer is cleared on successful connection or explicit `disconnect()`. Calling `disconnect()` sets `autoReconnect = false` to prevent further attempts.

WebSocket Protocol

All messages are JSON. Direction is relative to the frontend.

Server → Client

type	Fields	When
connected	sample_rate, n_channels, pre_trigger_seconds	Immediately after WebSocket open.

type	Fields	When
emg_chunk	data (2D array [nChannels][nSamples]), timestamp, n_samples	Every ~50 ms during streaming.
recording_status	recording (bool), message	After start/stop recording commands.
recording_saved	filepath, duration_seconds, n_channels, n_samples	After recording is saved to disk.
status	connected_to_trellis, recording, buffer_seconds, n_clients	In response to <code>get_status</code> .
error	message	On any server-side error.

Client → Server

command	Optional Fields	Effect
start_recording	metadata (object)	Captures pre-trigger buffer; begins accumulating samples.
stop_recording	timeline (object)	Stops recording; server saves <code>.pkl</code> file and broadcasts <code>recording_saved</code> .
get_status	—	Server responds with current status.

Channel Buffering

Each channel is backed by a `Float32Array` of length `bufferSize` (default 2048 — roughly 1 second at 2048 Hz). New chunks are appended using a shift-and-copy strategy:

When `nSamples < bufferSize`:

existing data → shift left

new samples → end

```
buffer.copyWithin(0, nSamples)
copy new samples into buffer[bufferSize - nSamples .. bufferSize - 1]
```

When `nSamples >= bufferSize`:

Replace entire buffer with the last `bufferSize` samples from the chunk.

`getChannelData(index)` returns the raw `Float32Array` reference (zero-copy). For a channel index out of range, it returns a zeroed buffer.

Recording API

startRecording(metadata?)

Sends { `command: 'start_recording'` } to the server. If `metadata` is provided, it is included in the message and persisted with the saved recording.

Metadata schema (all fields optional):

```
{
  "subjectId": "string",
  "sessionId": "string",
  "notes": "string",
  "sequenceName": "string",
  "totalItems": 0,
  "settings": {
    "pauseBetweenItems": 0.5,
    "restBetweenReps": 2.0,
    "prepTime": 1.0,
    "playbackSpeed": 1.0
  },
  "items": [
    { "index": 0, "model": "wrist.glb", "animation": 0, "repetitions": 3 }
  ],
  "startTime": "ISO 8601 timestamp"
}
```

stopRecording(timeline?)

Sends { `command: 'stop_recording'` }. If `timeline` is provided, the server stores it alongside the EMG data.

Timeline schema (all fields optional):

```
{
  "sessionStart": "ISO 8601",
  "sessionStartMs": 0,
  "sessionEnd": "ISO 8601",
  "totalDurationMs": 0,
  "events": [
    {
      "type": "phase_change",
      "time": "ISO 8601",
      "offsetMs": 0,
      "phase": "move",
      "item": 0,
      "rep": 0
    }
  ],
  "summary": {
    "totalTutorialTimeMs": 0,
    "totalPrepTimeMs": 0,
  }
}
```

```
    "totalMoveTimeMs": 0,
    "totalRestTimeMs": 0,
    "totalTransitionTimeMs": 0,
    "itemsCompleted": 0,
    "totalRepetitions": 0
  }
}
```

Event Callbacks

Register listeners using the `on*` methods. All callbacks are invoked synchronously.

Method	Callback signature	Triggered when
<code>onConnect(cb)</code>	<code>({ sampleRate, nChannels, preTriggerSeconds }) => void</code>	Server connection established and handshake received.
<code>onDisconnect(cb)</code>	<code>() => void</code>	WebSocket closes (before reconnect attempt).
<code>onData(cb)</code>	<code>({ data: Float32Array[], timestamp, nSamples }) => void</code>	New EMG chunk processed and buffers updated.
<code>onRecordingStatus(cb)</code>	<code>({ recording: boolean, message: string }) => void</code>	Recording state changes.
<code>onRecordingSaved(cb)</code>	<code>({ filepath, durationSeconds, nChannels, nSamples }) => void</code>	Server finishes writing a recording.
<code>onError(cb)</code>	<code>(message: string) => void</code>	Any error from server or local transport.

EMGChannelView

A DOM-based modal overlay that renders EMG waveforms on a single `<canvas>` element. Created once at startup and toggled via `show()/hide()/toggle()`.

Rendering Pipeline

- 1. Animation loop** — `requestAnimationFrame` drives `_drawChannels()` while the overlay is visible.
- 2. Per-channel rendering** (`_drawChannel`):
 - Fills a channel-sized rectangle with background color.
 - Draws a horizontal center gridline.
 - Labels the channel (`CH 1`, `CH 2`, etc.).
 - Reads the full `Float32Array` buffer from `EMGClient.getChannelData()`.
 - Computes per-buffer min/max for amplitude normalization.
 - Maps each pixel column to a sample index and plots the normalized waveform.
- 3. Color switching** — waveform color changes from green (`#00ff88`) to red (`#ff4444`) when `emgClient.isRecording` is `true`.

Canvas Dimensions

Parameter	Value	Notes
canvasWidth	800 px	Logical resolution; CSS scales to container width.
canvasHeight	600 px	
channelHeight	70 px	Per-channel strip height.
padding	10 px	Margin inside each strip.

Effective drawable height per channel: 60 px (70 – 10). Amplitude is scaled to 80% of half the drawable height to avoid clipping.

Pagination

- **8 channels per page** by default.
- Total pages derived from `nChannels / channelsPerPage` (recomputed on `onConnect`).
- Navigation via "Previous" / "Next" buttons.
- Page info label displays current channel range and page index (e.g., "Channels 9–16 (Page 2/8)").
- Buttons are visually dimmed (`opacity: 0.5`) and logically disabled at boundary pages.

Visual States

State	Indicators
Disconnected	Status badge: red, text "Disconnected".
Connected, idle	Status badge: green, text "Connected". Waveforms in green.
Connected, recording	Status badge: green. Red pulsing "RECORDING" banner. Waveforms in red.

The overlay can be dismissed by clicking outside the modal, pressing Escape, or clicking the close button.

Integration Points

App.js (Initialization)

`App.initEMG()` creates the client and view, connects, and wires the client to the sequence player:

```
this.emgClient = new EMGClient({ host: 'localhost', port: 8765, autoReconnect: true });
this.emgChannelView = new EMGChannelView(this.emgClient);
this.emgClient.connect();
this.sequencePlayer.onEMGClient = this.emgClient;
```

SequencePlayer (Automatic Recording)

- **Sequence start** → calls `emgClient.startRecording(metadata)` with trial metadata built from the current sequence configuration.
- **Sequence end / abort** → calls `emgClient.stopRecording(timeline)` with a finalized session timeline.

GUIView (Manual Controls)

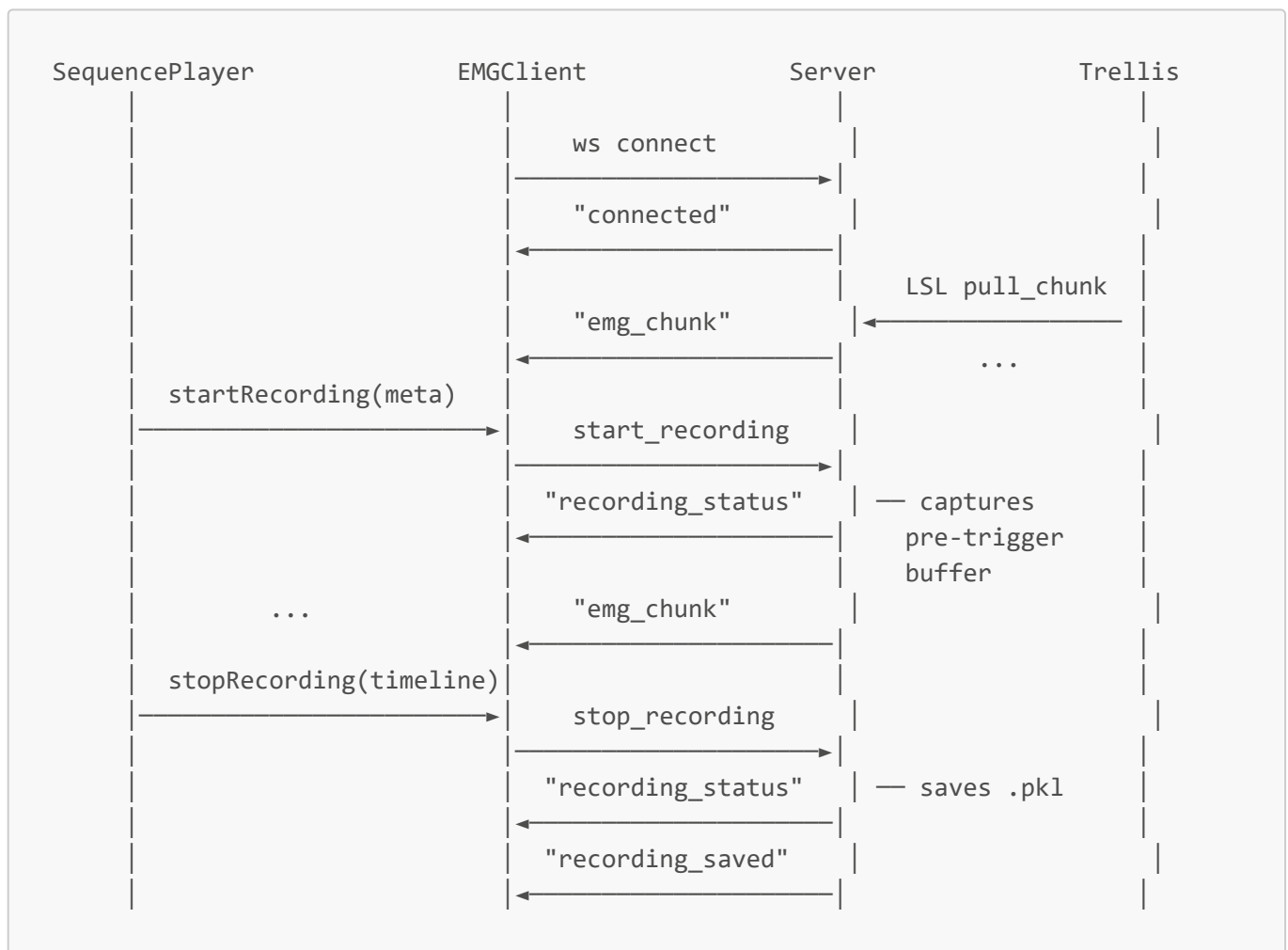
- "View Channels" button → `app.emgChannelView.show()`.
- "Start/Stop Recording" buttons → `app.emgClient.startRecording()` / `app.emgClient.stopRecording()`.

ProgressSidebar

- "View EMG" action → `app.emgChannelView.show()` in patient mode.

Data Flow

Sequence diagram for a typical recording session:



Recorded Data Format

The server writes Python pickle files (`.pkl`) with the following structure:

Key	Type	Description
data	np.ndarray [n_channels, n_samples]	Full EMG recording (pre-trigger + post-trigger).
srate	float	Sampling rate in Hz (typically 2048).
n_channels	int	Number of channels (typically 64).
filtered	bool	Whether bandpass/notch filtering was applied.
pre_trigger_seconds	float	Duration of pre-trigger buffer included.
timestamp	str	Recording timestamp (YYYYMMDD_HHMMSS).
trial_metadata	dict \ None	Metadata object from the frontend startRecording call.

Output location defaults to `./recordings/` relative to the server working directory.

Configuration Reference

Server CLI Arguments

```
python -m src.online.emg_websocket_server [OPTIONS]

--host TEXT           Server bind address      [default: localhost]
--port INT            Server port                [default: 8765]
--pre-trigger FLOAT   Pre-trigger buffer (s)      [default: 10.0]
--output TEXT         Output folder              [default: ./recordings]
--no-filter           Disable signal filtering
--chunk-interval INT  Chunk push interval (ms)   [default: 50]
```

Frontend Defaults

Constant	Value	Location
Buffer size	2048 samples	EMGClient.bufferSize
Reconnect interval	3000 ms	EMGClient.reconnectInterval
Channels per page	8	EMGChannelView.channelsPerPage
Canvas logical size	800 × 600	EMGChannelView.canvasWidth/Height
Channel strip height	70 px	EMGChannelView.channelHeight

Error Handling

Scenario	Behavior
Server unreachable	<code>onerror</code> fires; if <code>autoReconnect</code> is <code>true</code> , retries after <code>reconnectInterval</code> .
Unexpected disconnect	<code>onclose</code> fires; reconnection scheduled; <code>onDisconnect</code> callbacks invoked.
Malformed JSON from server	Caught in <code>_handleMessage</code> ; logged to console; message silently dropped.
Unknown message type	Logged as warning; no callback invoked.
Send while disconnected	<code>_send</code> returns <code>false</code> ; warning logged to console.
Start recording while already recording	Server responds with <code>error</code> message; client <code>onError</code> callbacks fire.
Stop recording with nothing in progress	Server responds with <code>error</code> message.
Channel index out of range	<code>getChannelData</code> returns a zeroed <code>Float32Array</code> of length <code>bufferSize</code> .

Known Constraints

- **Single-threaded rendering:** `EMGChannelView` draws all visible channels sequentially on one canvas. At 8 channels this is negligible, but extending to more channels per page may cause frame drops.
- Then again why the hell would you do that
- **No backpressure:** If the frontend cannot process chunks fast enough, messages will queue in the WebSocket receive buffer. The server does not implement flow control. We are not a company
- **Fixed normalization window:** Waveform amplitude is normalized against the entire buffer min/max on every frame. Transient spikes will compress the scale for all surrounding data until the spike exits the buffer.
- **In-memory pre-trigger buffer:** The server keeps `pre_trigger_seconds` worth of samples in a Python `deque`. At 64 channels × 2048 Hz × 10 s this is ~5 MB — manageable, but scales linearly with channel count and buffer duration.
- **Pickle output format:** Recordings are saved as `.pkl` files. This is fast and compatible with the project's analysis pipeline, but is not portable across Python versions or to other languages without conversion.
- **Hardcoded filter parameters:** Bandpass (20–500 Hz) and notch (50 Hz) filter cutoffs are set in the server source code, not exposed as configuration.